

자체 개발 Skill의 공식 지원 Skill 전환 연구

pub-wise-dev-std 플러그인 분석과 Claude Code · Cursor · Google Antigravity 공식 Skill 매핑 비교

대상 저장소 github.com/WisemanLim/pub-wise-dev-std (commit 4be11b9, 2026-08-28, v1.2.0)

비교 플랫폼 Claude Code · Cursor · Google Antigravity (보조: Codex · GitHub Copilot · Devin Desktop)

기준 일자 2026-10-01 (각 벤더 공식 문서 기준)

문서 유형 기술 백서 / 분석 보고서 + 전환 가이드

초록 (Abstract)

본 백서는 Claude Code 플러그인 `wise-dev-std`(공개 배포본 `pub-wise-dev-std`)가 제공하는 자체 Skill·Command·Agent·Hook을 정적으로 분석하고, Claude Code·Cursor·Google Antigravity가 2026년 10월 현재 공식적으로 제공하는 Skill 및 확장 메커니즘과 기능 단위로 매핑하여 비교한다. 분석 결과, 해당 플러그인은 9개 Skill·11개 Command·1개 Subagent·1개 SessionStart Hook·13개 스택 프로파일·13개 업종(KSIC) 도메인 오버레이로 구성되며, 이 중 스택 추천·스캐폴딩·업종 규제 오버레이·시험 디렉터리 규약은 어느 플랫폼에도 공식 대응물이 없는 고유 자산이다. 반면 코드 리뷰·보안 리뷰·UI 디자인·반복 구현 루프·문서(PDF/DOCX) 생성 영역은 Anthropic 공식 Skill(`frontend-design`, `webapp-testing`, `pdf`, `docx` 등)과 번들 Skill(`/code-review`, `/security-review`, `/loop`) 및 공식 플러그인(`feature-dev`, `pr-review-toolkit`, `security-guidance`, `ralph-loop`)으로 대체 또는 위임이 가능하다.

또한 2025-2026년에 걸쳐 업계가 Agent Skills 공개 표준(SKILL.md, agentskills.io) 과 AGENTS.md 로 수렴하면서, 플러그인이 현재 채택한 "AGENTS.md + IDE별 규칙 파일 복사(standardize)" 방식은 더 이상 최적이지 않다. Claude Code는 Custom Command를 Skill로 통합했고, Cursor는 2.4부터 SKILL.md를 지원하며 `/migrate-to-skills`를 제공하고, Antigravity는 Workflow를 2026-11-01까지 Skill로 대체(deprecate)한다. 본 문서는 이러한 변화를 근거로 ① 컴포넌트별 매핑 판정(대체·위임·유지·재설계), ② 공개 표준 준수 Skill 형식으로서의 전환 절차, ③ 단계별 로드맵과 검증 기준을 제시한다.

키워드: Agent Skills, SKILL.md, AGENTS.md, Claude Code Plugin, Cursor Rules, Antigravity, 개발환경 표준화, 스캐폴딩, KSIC 도메인 오버레이

목차

1. 서론
 2. 배경: 공식 확장 메커니즘의 수렴
 3. 대상 시스템 분석: pub-wise-dev-std
 4. 플랫폼별 공식 Skill·확장 메커니즘
 5. 매핑 비교분석
 6. 공식 지원 Skill 전환 가이드
 7. 전환 로드맵과 검증 기준
 8. 위험 요소와 한계
 9. 결론
- 부록 A. Anthropic 공식 Skill 목록
- 부록 B. 저장소 발견 이슈
- 부록 C. 참고문헌

1. 서론

1.1 연구 배경

AI 코딩 에이전트는 2025년 이후 "프롬프트 템플릿" 수준을 넘어 **패키지화된 절차 지식(Skill)** 을 배포·재사용하는 단계로 진입했다. **wise-dev-std**는 이 흐름에서 PRD(제품 요구사항 문서)와 최소한의 선택만으로 언어·패키지매니저·프레임워크·DB·실행방식을 추천하고 프로젝트 골격을 생성하는 Claude Code 플러그인이며, 표준을 **AGENTS.md**로 내보내 Cursor·Antigravity에서도 준용하도록 설계되었다.

그러나 플러그인이 설계된 시점 이후 각 플랫폼은 독자 규칙 포맷 대신 공통 Skill 표준을 채택했고, 일부 기능(코드 리뷰, UI 디자인, 문서 생성 등)은 벤더가 직접 공식 Skill로 제공하기 시작했다. 자체 Skill을 계속 유지할지, 공식 Skill로 대체할지, 혹은 공식 형식으로 재포장할지 판단할 근거가 필요하다.

1.2 연구 목적

1. **pub-wise-dev-std**의 전 구성요소를 기능 단위로 분해하여 역할과 의존성을 정리한다.
2. Claude Code·Cursor·Antigravity의 공식 Skill 및 확장 메커니즘(Rules, Skills, Commands/Workflows, Subagents, Hooks, Plugins)을 2026-10-01 기준으로 정리한다.
3. 자체 구성요소를 공식 대응물과 매핑하고, **대체 / 위임 / 유지 / 재설계** 판정을 내린다.
4. 자체 Skill을 공식 지원 형식으로 전환하는 구체적 가이드(파일 구조, frontmatter, 변환 예시, 로드맵)를 제공한다.

1.3 연구 범위와 방법

구분	내용
분석 대상	pub-wise-dev-std main 브랜치 (commit 4be11b9, 2026-08-28), 파일 200여 개 전수 목록화, Skill·Command·Agent·Hook·Script 원문 정독
비교 대상	Claude Code (code.claude.com), Anthropic Agent Skills (platform.claude.com, github.com/anthropics/skills), Cursor (cursor.com/docs), Google Antigravity (antigravity.google/docs)
보조 대상	OpenAI Codex, GitHub Copilot, Devin Desktop(구 Windsurf) — 교차 호환성 확인 용도
방법	정적 코드 분석 + 공식 문서 조사(2026-10-01 열람). 1차 출처로 확인되지 않은 사실은 "미확인"으로 표기
제외	플러그인 실제 실행 결과의 품질 평가, 성능 벤치마크, 상용 라이선스 법률 검토

2. 배경: 공식 확장 메커니즘의 수렴

2.1 Agent Skills 공개 표준 (SKILL.md)

Agent Skills는 Anthropic이 최초 개발하여 공개 표준으로 배포한 형식으로(agentskills.io), Skill 1개 = 디렉터리 1개이며 필수 파일 **SKILL.md**와 선택 디렉터리 **scripts/**, **references/**, **assets/**로 구성된다. 핵심 설계 원리는 **점진적 공개(progressive disclosure)**로, 시작 시에는 메타데이터(약 100 토큰)만 로드하고, 호출 시 본문(권장 5,000 토큰 미만), 필요 시에만 리소스를 읽는다.

필드	필수	제약 / 의미
name	예	1-64자, 소문자·숫자·하이픈, 상위 디렉터리명과 일치
description	예	1-1,024자, "무엇을 하고 언제 쓰는지" 기술 — 자동 호출의 트리거
license	아니오	라이선스 표기
compatibility	아니오	최대 500자, 실행 환경 요구사항
metadata	아니오	문자열→문자열 맵 (버전, 저자 등)
allowed-tools	아니오	공백 구분, 실험적(Experimental)

표 2-1. Agent Skills 표준 frontmatter (agentskills.io/specification)

agentskills.io는 Claude Code, Claude(앱·API), ChatGPT/Codex, Cursor, GitHub Copilot, VS Code, Gemini CLI, Junie, OpenCode, Roo Code, Kiro 등을 채택 클라이언트로 명시한다. Google Antigravity는 채택 목록에는 없으나 자체 문서에서 동일한 **SKILL.md** 형식을 사용한다.

2.2 AGENTS.md — 공통 상시(Always-on) 지침

AGENTS.md는 저장소 루트(및 하위 디렉터리)에 두는 도구 종립 지침 파일이다. Cursor·Antigravity·Codex·Copilot·Devin Desktop이 모두 읽으며, Claude Code도 v2.1.277부터 기본 모드(**claude-md-or-agents-md**)에서 **CLAUDE.md**가 없을 때 **AGENTS.md**를 네이티브로 읽는다. **CLAUDE.md**가 이미 있으면 **AGENTS.md**는 읽지 않으므로, 교차 도구 저장소에서는 **CLAUDE.md**에 **@AGENTS.md** 임포트를 두는 것이 공식 권장 방식이다.

2.3 Command·Workflow의 Skill 통합

플랫폼	변화	근거
Claude Code	Custom Command가 Skill로 통합. <code>.claude/commands/x.md</code> 와 <code>.claude/skills/x/SKILL.md</code> 모두 /x 생성. 신규 플러그인은	code.claude.com/docs/en/skills-plugins-reference

플랫폼	변화	근거
	skills/ 권장	
Cursor	2.4(2026-01-22) Agent Skills 지원. /migrate-to-skills로 동적 규칙·커맨드를 Skill로 변환. commands 문서가 skills 페이지로 리다이렉트	cursor.com/changelog/2-4, docs/context/skills
Antigravity	Workflow는 2026-11-01까지 Agent Skills로 대체(deprecated). /migrate-workflows 제공	antigravity.google/docs/ide/workflows
Devin Desktop	Workflow → Skill 마이그레이션 권장	docs.devin.ai/desktop/cascade/workflows

표 2-2. 플랫폼별 Command/Workflow → Skill 수렴 현황

2.4 플러그인 패키징

Skill 묶음을 배포하는 단위는 플랫폼마다 다르다. Claude Code는 `.claude-plugin/plugin.json` + `marketplace.json`, Cursor는 `.cursor-plugin/plugin.json`(2.5부터 Marketplace) 또는 공개 **Agent Plugins** 형식, Antigravity는 `plugin.json` + `skills/·agents/·rules/·hooks.json` 구조를 쓴다. 2026-08-06 공개된 Agent Plugins 1.0(agent-plugins.org)은 Codex·Cursor·Copilot·Kiro·VS Code가 참여했으나 Anthropic·Google은 참여하지 않았다. 따라서 패키지 형식은 아직 분화되어 있고, Skill(SKILL.md) 단위만 사실상 통일되었다고 보는 것이 정확하다.

3. 대상 시스템 분석: pub-wise-dev-std

3.1 개요

`pub-wise-dev-std`는 단일 마켓플레이스(`wise-dev-std`)에 단일 플러그인(`wise-dev-std`, v1.2.0)을 담은 공개 배포본이다. 목적은 신규 프로젝트 착수 시 반복되는 의사결정 — 환경 구성, 패키지매니저 선택, `local/dev/staging/prod` 분리, 실행 방식, 업종 규제 반영 — 을 표준화·자동화하는 것이다. "Claude CLI 1차 구현, Cursor·Antigravity 준용"을 명시한다.

3.2 핵심 파이프라인

```
PRD.md → 스택 추천 → 골격 생성 → 환경 파일 → 구현 + 시험(자동 반복)
  prd      recommend  scaffold    env-init      implement
```

선택: `standardize` · `review` · `reverse-prd` · `req-update` · `test` · `ui-design`

그림 3-1. `wise-dev-std` 표준 흐름

3.3 구성요소 목록

유형	이름	역할	규모
Skill	<code>prd-advisor</code>	5질문(Why·Who·What·How·Success) 설문 → One-Page/폴스펙 PRD, KSIC 기반 KPI·NFR 제안	168줄
Skill	<code>stack-advisor</code>	<code>profiles</code> × <code>domains</code> 결정 매트릭스로 언어·PM·FW·DB·실행방식 추천, <code>trends-cache</code> 우선 + WebSearch	223줄
Skill	<code>project-scaffolder</code>	정적 템플릿 복사·치환으로 Makefile·compose·매니페스트·CI· <code>.env</code> ·test/·COMPLIANCE.md 생성 (+references 2개)	322줄
Skill	<code>test-runner</code>	시나리오→실행→수정·재시험→결과, <code>test/dev-env</code> , <code>test/impl/<Nth></code> 규약, 청킹	167줄
Skill	<code>code-reviewer</code>	독자 레벨(0-4) 라인 단위 리뷰 → <code>.review/<name>/</code> 트리	165줄
Skill	<code>depth-reviewer</code>	스택·라이선스·보안·유지보수·아키텍처·ISO 5230 법적 리스크, 5단계 등급	155줄
Skill	<code>living-doc</code>	추가 요구사항을 PRD·README·COMPLIANCE 등 비소스 문서에 전파	151줄
Skill	<code>reverse-prd</code>	기존 소스에서 PRD 역도출, 미확인 항목 표기	154줄
Skill	<code>ui-design-advisor</code>	레퍼런스 사이트·디자인 시스템·접근성 스택 델타	175줄
Command	<code>prd / recommend / scaffold / env-init / implement</code>	핵심 5단계 파이프라인 진입점	22-208줄
Command	<code>standardize / review / reverse-prd / req-update / test / ui-design</code>	선택 명령	19-50줄

유형	이름	역할	규모
Agent	stack-architect	recommend→scaffold→env-init→implement 독립 수행 서브에이전트	44줄
Hook	SessionStart → detect-prd.sh	PRD.md 유무에 따라 1줄 안내, 스캐폴딩 흔적 있으면 침묵	27줄
Data	profiles/*.yaml (13+custom)	node-next-nest, python-fastapi, go-gin, rust-axum, java-spring, csharp-dotnet, cpp-cmake, bio-rag-research, ios-swiftui, android-compose, flutter-app, react-native-app	—
Data	domains/*.yaml (13)	KSIC 대분류별 규제·데이터등급·선호스택·인프라·시험 오버레이	—
Data	templates/	AGENTS.md, gitignore 조각, scaffold/<profile>/ 정적 템플릿	—
Script	install-portable.sh 외 3종	멀티-IDE 규칙 파일 설치·zip 패키징, 트렌드 캐시 신선도 점검	—

표 3-1. 플러그인 구성요소 전수 목록

3.4 설계 특성

- **데이터 주도 확장성:** 스택은 `profiles/*.yaml`, 업종은 `domains/*.yaml` 1파일 추가만으로 확장되며 코드 수정이 필요 없다. 이는 Skill 표준의 `references/·assets/` 개념과 정확히 대응한다.
- **Command-Skill 이원 구조:** Command는 인자 파싱과 절차, Skill은 지식 베이스를 담는다. Claude Code가 Command를 Skill로 통합한 현재 기준에서는 중복 계층이다.
- **플러그인 루트 변수 의존:** `#{CLAUDE_PLUGIN_ROOT}` — Skill 4개·Command 4개가 이 변수로 `profiles/domains/templates`를 참조한다. Claude Code 외 플랫폼에서는 해석되지 않는다.
- **안전 규칙 내장:** 설치·네트워크 명령 금지, 실시크릿 금지, `.env.prod` 키 목록만, 기존 파일 `*.generated` 보존 등이 각 Skill에 명문화되어 있다.
- **공식 표준 적합성:** 9개 Skill 모두 `name` = 디렉터리명, `description` 261-557자(한도 1,024자), 본문 151-322 줄(권장 500줄 미만)로 **Agent Skills 사양을 이미 충족**한다. 전환 비용이 낮다.

3.5 멀티-IDE 내보내기(standardize)의 현황

생성 파일	대상 도구	2026-10 공식 기준 평가
AGENTS.md	Claude Code·Codex·Cursor·Antigravity	유효. 단, Claude Code는 CLAUDE.md 부재 시에만 읽음
.cursor/rules/wise-dev-std.mdc (alwaysApply)	Cursor	유효하나 AGENTS.md와 내용 중복 → 컨텍스트 이중 소모
.antigravity/rules.md	Antigravity	공식 경로 아님. 공식은 AGENTS.md/GEMINI.md 또는 .agents/rules/*.md
.windsurf/rules/wise-dev-std.md	Windsurf	하위호환 유효. 현행 Devin Desktop은 .devin/rules/ 우선
.github/copilot-instructions.md	GitHub Copilot	유효. Copilot은 AGENTS.md도 읽으므로 중복

생성 파일	대상 도구	2026-10 공식 기준 평가
GEMINI.md	Gemini CLI	Gemini CLI 무료 티어는 2026-06-18 Antigravity CLI로 대체. GEMINI.md는 Antigravity에서도 유효
.clinerules, .roo/rules/	Cline, Roo Code	유효 (본 연구 범위 외)

표 3-2. standardize 산출물의 공식 경로 적합성

핵심 문제는 standardize가 "규칙(상시 지침)"만 내보내고 "Skill(절차 지식)"은 내보내지 않는다는 점이다. 그 결과 Cursor·Antigravity 사용자는 언어·PM 표준은 따르지만 **recommend·scaffold·implement** 같은 핵심 절차를 실행할 수 없다. 2026년 현재 세 플랫폼 모두 SKILL.md를 지원하므로, 이 한계는 Skill 자체를 배포하는 방식으로 해소할 수 있다.

4. 플랫폼별 공식 Skill·확장 메커니즘

4.1 Claude Code

4.1.1 SKILL.md 확장 필드

Claude Code는 표준 필드 외에 다음 확장 필드를 지원한다(모두 선택). 이 필드들은 다른 플랫폼에서 무시되므로, 이식성이 필요한 Skill은 표준 필드 + 본문만으로 동작해야 한다.

필드	용도
when_to_use	description에 덧붙는 트리거 문구 (합계 1,536자 상한)
argument-hint, arguments	자동완성 힌트, 이름 있는 위치 인자(\$name)
disable-model-invocation	true = 사용자만 호출 (기존 Command와 동일한 동작)
user-invocable	false = / 메뉴에서 숨김, 모델 전용 지식 베이스
allowed-tools, disallowed-tools	호출 토큰의 도구 사전 승인/제거
context: fork, agent, background	격리된 서브에이전트에서 실행 (Explore/Plan/general-purpose 등)
model, effort	모델·추론 노력 수준 지정
hooks, paths, shell	Skill 실행 중 후, glob 활성 범위, 셸 종류

표 4-1. Claude Code 전용 SKILL.md 필드 (code.claude.com/docs/en/skills)

치환 변수는 \$ARGUMENTS, \$N, \${CLAUDE_SKILL_DIR}, \${CLAUDE_PLUGIN_ROOT}, \${CLAUDE_PLUGIN_DATA}, \${CLAUDE_PROJECT_DIR} 등이며, 느낌표+백틱(!cmd) 구문으로 Skill 로드 전 셸 결과를 주입할 수 있다. Skill 위치는 개인(~/.claude/skills/), 프로젝트(.claude/skills/), 플러그인(<plugin>/skills/, /plugin:skill로 호출) 등이다. Claude Code는 .agents/skills/를 읽지 않는다는 점이 교차 배포 설계의 제약이다.

4.1.2 공식 Skill·번들 Skill·공식 플러그인

구분	항목	개발 워크플로 관련성
anthropics/skills (19종)	docx, pdf, pptx, xlsx, skill-creator, mcp-builder, webapp-testing, frontend-design, web-artifacts-builder, doc-coauthoring, theme-factory, brand-guidelines, internal-comms, canvas-design, algorithmic-art, slack-gif-creator, claude-api, academy-guide, discernment-nudge	문서 산출(docx/pdf), E2E 시험(webapp-testing), UI(frontend-design), 문서 공동작성(doc-coauthoring), Skill 제작·평가(skill-creator)
번들 Skill	/code-review(/review), /simplify, /batch, /loop, /run, /verify, /debug, /claude-api	PR 리뷰, 대규모 변경 분할, 반복 실행, 실제 앱 구동 검증
내장 명령	/init, /security-review, /schedule, /import [codex gemini cursor], /skills, /skill-doctor	CLAUDE.md 생성, 보안 리뷰, 타 도구 설정 가져오기

구분	항목	개발 워크플로 관련성
공식 플러그인 (anthropics/claude-code)	code-review, feature-dev, pr-review-toolkit, security-guidance, frontend-design, commit-commands, hookify, plugin-dev, ralph-wiggum(ralph-loop), agent-sdk-dev	기능 개발 워크플로, 다중 에이전트 PR 리뷰, 보안 가드, 자기참조 반복 루프
공식 마켓플레이스 (claude-plugins-official)	315개 항목. Anthropic 작성: claude-md-management, code-simplifier, claude-security, code-modernization, skill-creator, session-report 등 + LSP 12종	문서 관리, 보안, 현대화

표 4-2. Claude Code 공식 제공 Skill 생태계

참고: `docx`·`pdf`·`pptx`·`xlsx`는 Apache 2.0이 아닌 **소스 공개(source-available)** 라이선스이며, Claude API의 사전 탑재 Skill과 달리 Claude Code에서는 `/plugin marketplace add anthropics/skills` 후 `document-skills@anthropic-agent-skills`를 설치해야 한다.

4.2 Cursor

메커니즘	위치 / 형식	비고
Rules	<code>.cursor/rules/*.mdc</code> (description, globs, alwaysApply)	Always / Apply Intelligently / Specific Files / Manually 4종. 500줄 미만 권장
AGENTS.md	루트 + 중첩 디렉터리	하위 파일이 상위와 결합, 구체적인 쪽 우선. CLI는 CLAUDE.md도 읽음
Agent Skills	<code>.cursor/skills/</code> , <code>.agents/skills/</code> , 호환: <code>.claude/skills/</code> , <code>.codex/skills/</code>	2.4(2026-01-22) 도입. frontmatter: name, description, paths, disable-model-invocation, icon, color, metadata
Commands	<code>.cursor/commands/*.md</code>	1.6 도입, 현재 Skill로 사실상 흡수(/migrate-to-skills)
Subagents	<code>.cursor/agents/</code> , 호환: <code>.claude/agents/</code> , <code>.codex/agents/</code>	name, description, model, readonly, is_background
Hooks	<code>.cursor/hooks.json</code> (version 1)	sessionStart, preToolUse, afterFileEdit, stop 등 20여 이벤트
MCP	<code>.cursor/mcp.json</code>	stdio / SSE / Streamable HTTP
Plugins	<code>.cursor-plugin/plugin.json</code> 또는 Agent Plugins	2.5(2026-02-17) Marketplace, /add-plugin
Bugbot	<code>.cursor/BUGBOT.md</code>	GitHub/GitLab 등 PR 자동 리뷰 + Autofix

표 4-3. Cursor 공식 확장 메커니즘

Cursor는 Anthropic처럼 범용 공식 Skill 카탈로그를 별도 저장소로 배포하지 않는다(미확인 — 1차 출처에서 확인되지 않음). 대신 `.claude/skills/`와 `.claude/agents/`를 **하위호환 경로로 읽기 때문에**, Claude Code용 Skill을 거의 그대로 재사용할 수 있다는 점이 가장 중요한 공식 특성이다.

4.3 Google Antigravity

Antigravity는 2026-05-19 2.0(에이전트 우선 데스크톱 앱 + CLI + SDK)으로 개편되었고, 현행 문서는 출시 초기의 `agent/` 대신 `.agents/` 디렉터리를 사용한다.

메커니즘	위치 / 형식	비고
Rules	AGENTS.md, GEMINI.md, .agents/rules/*.md, 전역 ~/.gemini/...	트리거: always_on / model_decision / glob / manual. 파일당 24KB, 합계 20k 토큰
Workflows	Customizations 패널에서 생성, /workflow-name	12,000자 제한. 2026-11-01까지 Skill로 대체, /migrate-workflows
Skills	.agents/skills/<skill>/SKILL.md, 전역 ~/.gemini/config/skills/	description 필수, name은 폴더명 기본. 보조 폴더: scripts/, examples/, resources/
Subagents	.agents/agents/<name>.md	name, description, tools, model(inherit/flash/pro), skills 등
Hooks	.agents/hooks.json	PreToolUse, PostToolUse, PreInvocation, PostInvocation, Stop (SessionStart 없음)
MCP	.agents/mcp_config.json	stdio, Streamable HTTP/SSE, WebSocket, MCP Store
Plugins	plugin.json + skills/agents/rules/hooks.json	.agents/plugins/, agy plugin ..., Marketplace UI
내장 명령	/plan, /goal, /grill-me, /learn, /browser, /schedule, /plugin 등	계획·목표 반복·인터뷰·브라우저 검증
Artifacts	계획·diff·다이아그램·브라우저 녹화	인라인 피드백(Human-in-the-loop)

표 4-4. Google Antigravity 공식 확장 메커니즘

4.4 교차 플랫폼 매트릭스

메커니즘	Claude Code	Cursor	Antigravity
상시 지침	CLAUDE.md (AGENTS.md는 CLAUDE.md 부재 시)	AGENTS.md (중첩)	AGENTS.md / GEMINI.md
범위 규칙	.claude/rules/*.md (paths:)	.cursor/rules/*.mdc	.agents/rules/*.md (trigger)
Skill	.claude/skills/, 플러그인 skills/	.cursor/skills/, .agents/skills/, .claude/skills/	.agents/skills/
명령/워크플로	Skill로 통합 (commands/ 레거시)	Skill로 흡수	Workflow → Skill (11/1 폐기)
서브에이전트	.claude/agents/	.cursor/agents/, .claude/agents/	.agents/agents/
훅	hooks/hooks.json (SessionStart 등)	.cursor/hooks.json (sessionStart)	.agents/hooks.json (SessionStart 없음)
패키지	.claude-plugin/ + marketplace	.cursor-plugin/ / Agent Plugins	plugin.json (.agents/plugins/)

표 4-5. 메커니즘 × 플랫폼 경로 매트릭스

시사점

- ① Skill 본문은 1벌로 3개 플랫폼에서 공유 가능하다. 공유 경로는 Cursor 기준 `.claude/skills/` 또는 `.agents/skills/`이지만, Claude Code는 `.agents/`를, Antigravity는 `.claude/`를 읽지 않으므로 **정본 1벌 + 심볼릭 링크(또는 복사)**가 필요하다.
- ② 이식 가능한 frontmatter는 `name, description, license, compatibility, metadata`(+실험적 `allowed-tools`)뿐이다.
- ③ 혹·플러그인 패키지는 플랫폼별로 별도 작성해야 한다.

5. 매핑 비교분석

5.1 판정 기준

판정	정의	조치
대체 (Replace)	공식 Skill/기능이 동등 이상의 기능을 제공	자체 구현 제거, 공식 Skill 호출로 전환
위임 (Delegate)	일부 하위 기능이 공식과 중복	중복부는 공식 Skill에 위임, 고유부만 유지
유지 (Keep)	공식 대응물 없음 — 고유 자산	공개 표준 형식으로 재포장만 수행
재설계 (Redesign)	설계 전제가 공식 변화로 무효화	구조 자체를 재설계

표 5-1. 매핑 판정 기준

5.2 컴포넌트별 매핑 총괄표

자체 구성요소	Claude Code 공식 대응	Cursor 공식 대응	Antigravity 공식 대응	판정
prd-advisor + /prd	doc-coauthoring (구조화 공동작성), AskUserQuestion	없음 (Plan 모드)	/grill-me(인터뷰), /plan	유지 — KSIC KPI·NFR 제안이 고유. 설문 흐름은 doc-coauthoring 패턴 참고
stack-advisor + /recommend	없음 (Plan 에이전트는 범용)	없음	없음	유지 — 핵심 IP. 데이터를 references/로 이동
project-scaffolder + /scaffold	/init은 CLAUDE.md만 생성 — 대응 없음	없음	없음	유지 — 복사·치환 로직을 scripts/로 결정화
/env-init	없음	없음	없음	유지 — Command → Skill 전환
test-runner + /test	webapp-testing(Playwright), /run, /verify	Browser 서브에이전트	/browser 서브에이전트	위임 — test/ 규약 유지, E2E·UI 검증은 공식 위임
/implement	feature-dev 플러그인, ralph-loop, /loop, /batch	Agent/Background Agent	/goal, /plan	위임 — 반복 엔진은 공식, PRD Epic· 차수 규약 유지
code-reviewer	/code-review, code- review·pr-review-toolkit 플러그인, /simplify	Bugbot	없음 (Artifacts 리뷰)	위임 — PR 리뷰는 공식 대체, 레벨(0-4) 교육형 리뷰만 유지
depth-reviewer	/security-review, security- guidance, claude-security	Bugbot (보안 규칙)	없음	위임 — 보안은 공식, 라이선스·ISO 5230·상용화 등급 유지
review --pdf	pdf, docx Skill	없음	없음	대체 — pandoc 의존 제거, 공식 문서 Skill 사용
ui-design-advisor + /ui-design	frontend-design, theme-factory, brand-guidelines	없음	없음	위임 — 미학·구현은 frontend-design, 국내 레퍼런스·KWAG 델타만 유지
living-doc + /req-update	claude-md-management(부분), doc-coauthoring	없음	없음	유지
reverse-prd	/init(코드 분석→CLAUDE.md, 부분)	없음	없음	유지

자체 구성요소	Claude Code 공식 대응	Cursor 공식 대응	Antigravity 공식 대응	판정
/standardize	AGENTS.md 네이티브 지원, /init 규칙 임포트, /import	AGENTS.md·.claude/skills 네이티브 판독	AGENTS.md 네이티브, .agents/skills	재설계 — 규칙 복사 → Skill 배포기
stack-architect (Agent)	서브에이전트 .claude/agents/	.claude/agents/ 호환 판독	.agents/agents/	유지 — 경로만 이식
SessionStart 혹	hooks.json SessionStart	.cursor/hooks.json sessionStart	SessionStart 없음 → AGENTS.md 문구로 대체	유지/대체
plugin + marketplace	.claude-plugin/	.cursor-plugin/ 또는 Agent Plugins	plugin.json	유지 + 추가 패키지

표 5-2. 자체 구성요소 ↔ 공식 Skill/기능 매핑

5.3 영역별 상세 분석

5.3.1 고유 자산 영역 (유지)

stack-advisor·project-scaffolder·env-init·domains/ 는 세 플랫폼 어디에도 공식 대응물이 없다. 공식 Skill 카탈로그는 범용 생산성(문서·디자인·테스트·리뷰)에 집중되어 있고, "PRD → 스택 결정 → 표준 골격"이라는 의사결정 지식, 특히 **한국 규제(KSIC 업종별 전자금융감독규정·의료법·ISMS-P 등)를 1순위로 반영하는 도메인 오버레이**는 벤더가 제공할 가능성이 낮은 조직 고유 지식이다. 이 영역은 기능을 바꾸지 않고 형식만 표준화하는 것이 합리적이다.

5.3.2 중복 영역 (위임)

리뷰 계열: Claude Code **/code-review**는 effort 수준(low~max/ultra), **--fix**(자동 수정), **--comment**(PR 코멘트)를 제공하고, pr-review-toolkit은 주석·테스트·에러처리·타입설계 전문 에이전트를 제공한다. 자체 code-reviewer의 범용 결함 탐지는 이들과 중복된다. 다만 **독자 레벨(0=비개발자 ~ 4=아키텍트)에 맞춘 라인 단위 설명과 .review/ 트리 산출물**은 인수인계·교육 목적의 고유 기능이므로 유지한다. depth-reviewer의 보안 단계는 **/security-review·security-guidance**로 위임하고, 라이선스 감사·ISO 5230·5단계 상용화 등급은 유지한다.

구현 루프: **/implement**의 "구현→시험→수정→재시험(최대 5회)" 반복은 ralph-loop(자기참조 반복), **/loop**, Antigravity **/goal**이 공식으로 제공하는 패턴이다. 반면 PRD Epic 파싱, 계획표, **test/impl/<Nth>/** 차수 규약, BLOCKED 묶음 질문, README(한/영) 현행화는 고유하다. 반복 엔진은 플랫폼 공식 기능에 맡기고 Skill은 "한 Epic 처리 절차"에 집중시키면 플랫폼 간 이식성이 높아진다.

UI 디자인: frontend-design은 "템플릿 기본값을 피하는 의도적 미학"을 구현 수준에서 지도한다. 자체 ui-design-advisor의 글로벌 트렌드·컴포넌트 예시는 이와 중복되며, 국내 레퍼런스 사이트, KWCAG 2.2 접근성, 플랫폼별 (SwiftUI/Compose/Flutter) 디자인 스택 델타만 차별점이다.

문서 산출: depth-reviewer·code-reviewer의 PDF 변환은 pandoc 의존 셸 절차로 기술되어 있다. 공식 pdf·docx Skill은 생성·병합·양식 채우기·검증 절차까지 포함하므로 대체가 바람직하다.

5.3.3 재설계 영역 (standardize)

standardize의 설계 전제는 "각 IDE가 서로 다른 규칙 포맷을 쓰므로 동일 내용을 N개 파일로 복사한다"였다. 2026년 현재 이 전제는 두 가지 이유로 무효화되었다. 첫째, Cursor·Antigravity·Copilot·Codex·Devin Desktop이 모두 AGENTS.md를 네이티브로 읽으므로 복사본은 **컨텍스트 중복 소모**만 낳는다. 둘째, 세 플랫폼이 모두 SKILL.md를 지원하므로 **규칙뿐 아니라 절차(Skill) 자체를 배포**할 수 있다. 따라서 standardize는 "AGENTS.md 1개 + Skill 정본 배포 + 플랫폼별 링크/패키지 생성기"로 재설계되어야 한다.

5.4 정량 요약

판정	개수	구성요소
유지	7	prd-advisor, stack-advisor, project-scaffolder, env-init, living-doc, reverse-prd, stack-architect
위임	5	test-runner, implement, code-reviewer, depth-reviewer, ui-design-advisor
대체	1	review --pdf 변환
재설계	1	standardize (+ install-portable.sh)
패키지/훅	2	plugin·marketplace (추가 패키지), SessionStart 훅 (플랫폼별)

표 5-3. 판정 분포

6. 공식 지원 Skill 전환 가이드

6.1 전환 원칙

- Skill 단일화:** `commands/*.md`를 모두 `skills/<name>/SKILL.md`로 통합한다. 사용자 진입점이던 Command는 `disable-model-invocation: true`로, 지식 베이스 Skill은 `user-invocable: false`로 구분한다(Claude Code 전용 필드이며 타 플랫폼에선 무시되어도 동작에 지장 없음).
- 표준 필드 우선:** 이식에 필요한 정보는 `name·description·metadata·compatibility`에만 담는다. 확장 필드는 "있으면 좋은" 최적화로만 사용한다.
- 경로 자립화:** `${CLAUDE_PLUGIN_ROOT}/profiles/...` 대신 Skill 디렉터리 기준 상대 경로 (`references/profiles/...`)를 사용한다. Claude Code에서는 `${CLAUDE_SKILL_DIR}`로 보강한다.
- 데이터는 references/·assets/로:** `profiles·domains·trends-cache`는 `references/`, 정적 스캐폴드 템플릿과 gitignore 조각은 `assets/`로 옮긴다.
- 결정적 작업은 scripts/로:** 템플릿 복사·`{{PROJECT_NAME}}` 치환·gitignore 조립·차수 계산처럼 매번 같은 결과여야 하는 작업은 LLM 지시가 아닌 스크립트로 둔다(출력만 컨텍스트에 들어가 토큰 절감).
- 중복은 공식 Skill에 위임:** 5장의 위임 판정 항목은 본문에서 공식 Skill 이름을 명시적으로 호출한다.

6.2 목표 구조

```

pub-wise-dev-std/
├── .claude-plugin/marketplace.json      # Claude Code 배포 (유지)
├── plugins/wise-dev-std/
│   ├── .claude-plugin/plugin.json      # Claude Code 플러그인
│   ├── .cursor-plugin/plugin.json      # (신규) Cursor 플러그인
│   └── plugin.json                     # (신규) AntigraVity / Agent Plugins
├── skills/                             # ★ 정본 — 모든 플랫폼 공용
│   ├── wds-prd/ SKILL.md               references/prd-templates.md
│   ├── wds-recommend/ SKILL.md         references/{profiles,domains}/ trends-cache.yaml
│   ├── wds-scaffold/ SKILL.md          scripts/scaffold.sh assets/{scaffold,gitignore}/
│   ├── wds-env-init/ SKILL.md          scripts/env-init.sh
│   ├── wds-implement/ SKILL.md
│   ├── wds-test/ SKILL.md              scripts/next-iteration.sh
│   ├── wds-review/ SKILL.md            references/{depth,code}-method.md
│   ├── wds-living-doc/ SKILL.md
│   ├── wds-reverse-prd/ SKILL.md
│   ├── wds-ui-design/ SKILL.md
│   └── wds-standardize/ SKILL.md       scripts/install.sh assets/AGENTS.md
├── agents/stack-architect.md
├── hooks/hooks.json                   # Claude Code SessionStart
└── README.md

```

그림 6-1. 전환 후 디렉터리 구조(안)

Skill 이름에 `wds-` 접두사를 붙이는 이유는, 플러그인 네임스페이스(`/wise-dev-std:recommend`)가 없는 Cursor·Antigravity 프로젝트 설치 시 `recommend·test·review` 같은 일반 이름이 다른 Skill이나 내장 명령(`/review`)과 충돌하기 때문이다.

6.3 Frontmatter 변환 예시

6.3.1 지식 베이스 Skill (stack-advisor → wds-recommend로 통합)

변환 전: Command(`commands/recommend.md`)와 Skill(`skills/stack-advisor/SKILL.md`)이 분리되어 있고 데이터는 `${CLAUDE_PLUGIN_ROOT}`로 참조한다.

```
# commands/recommend.md (변환 전)
---
description: "PRD.md와 기본 선택으로 트렌드에 맞는 스택 추천 / ..."
argument-hint: "[프로파일힌트|node|python|...] [업종|...] [--trends]"
allowed-tools: Read, Glob, Grep, WebSearch
... ${CLAUDE_PLUGIN_ROOT}/profiles/*.yaml 를 읽고 stack-advisor 스킬 적용 ...
```

변환 후: 하나의 Skill로 통합하고, 표준 필드로 이식성을 확보하며 Claude Code 확장 필드를 덧붙인다.

```
# skills/wds-recommend/SKILL.md (변환 후)
---
name: wds-recommend
description: >
  PRD.md와 기본 선택(언어·FE/BE·팀 규모·업종 KSIC)으로 언어·패키지매니저·
  프레임워크·DB·실행방식을 추천한다. Recommend a trend-aligned stack from PRD.md.
  "스택 추천", "기술스택 결정", "stack recommendation" 요청 시 사용.
license: Internal
compatibility: Claude Code, Cursor 2.4+, Antigravity 2.0+. WebSearch 선택.
metadata:
  version: "2.0.0"
  source: "github.com/WisemanLim/pub-wise-dev-std"
# --- 이하 Claude Code 확장 (타 플랫폼은 무시) ---
argument-hint: "[profile-hint] [domain] [--trends]"
allowed-tools: Read Glob Grep WebSearch
---

## 절차
1. 이 Skill 디렉터리의 references/profiles/*.yaml, references/domains/*.yaml 을 읽는다.
2. PRD.md (루트 또는 docs/) 를 탐색·분석한다. ...
```

6.3.2 사용자 진입 Skill (env-init)

```
---
name: wds-env-init
description: >
  local/dev/staging/prod 환경 파일과 compose override, DB(sqlite/postgres)를
  생성한다. Generate env files for 4 environments. "환경 파일", "env-init" 요청 시.
metadata: { version: "2.0.0" }
disable-model-invocation: true      # Claude Code: 사용자만 호출 (기존 Command 동작 보존)
argument-hint: "<profile-id> [--db sqlite|postgres]"
---
```

1. ``bash scripts/env-init.sh "$ARGUMENTS"`` 를 실행해 파일을 생성한다.
2. 출력된 생성 목록을 보고하고, `.env.prod` 는 키 목록만 있음을 확인한다.

6.4 공식 Skill 위임 패턴

위임 판정 Skill의 본문은 공식 Skill이 설치되어 있으면 호출하고, 없으면 자체 최소 절차로 풀백하도록 작성한다. 이렇게 하면 공식 Skill이 없는 플랫폼(Antigravity 등)에서도 동작이 보장된다.

```
# skills/wds-review/SKILL.md (발체)
## 1. 결함·보안 리뷰 — 공식 위임
- Claude Code: /code-review high 를 실행하고, 보안은 /security-review 결과를 병합한다.
- Cursor: Bugbot 결과(.cursor/BUGBOT.md 규칙)가 있으면 인용한다.
- 위 기능이 없으면 references/code-method.md 의 최소 체크리스트로 수행한다.
## 2. 고유 산출물 — 자체 수행
- 독자 레벨(0-4) 라인 단위 설명 → .review/CODE-REVIEW-Lv<N>/
- 라이선스·ISO 5230·5단계 상용화 등급 → .review/REVIEW-InDepth.md
## 3. 문서화 — 공식 위임
- --pdf 요청 시 pdf Skill(또는 docx Skill)로 변환한다. pandoc 직접 호출 금지.
```

자체 기능	위임 대상 공식 Skill	설치
E2E/UI 시험 (test-runner)	webapp-testing	/plugin install example-skills@anthropic-agent-skills
PDF/DOCX 보고서	pdf, docx	/plugin install document-skills@anthropic-agent-skills
UI 미학·구현 (ui-design)	frontend-design	/plugin install frontend-design@claude-plugins-official
PR 결함 리뷰 (code-reviewer)	/code-review (번들), pr-review-toolkit	번들 /pr-review-toolkit@claude-plugins-official
보안 (depth-reviewer)	/security-review, security-guidance	내장 /security-guidance@claude-plugins-official
반복 구현 루프 (implement)	ralph-loop, /loop; Antigravity /goal	ralph-loop@claude-plugins-official / 내장
기능 개발 단계 (implement)	feature-dev	feature-dev@claude-plugins-official
Skill 품질 평가	skill-creator (eval·benchmark)	example-skills@anthropic-agent-skills

표 6-1. 위임 대상 공식 Skill과 설치 경로 (Claude Code 기준)

6.5 standardize 재설계

재설계된 `wds-standardize`는 다음 세 가지만 수행한다.

1. **AGENTS.md 1개 생성**(정본 상시 지침). Claude Code를 위해 `CLAUDE.md`가 있으면 `@AGENTS.md` 한 줄을 추가하고, 없으면 그대로 둔다(네이티브 판독).
2. **Skill 정보 설치**: 대상 프로젝트의 `.agents/skills/wds-*`에 복사(Cursor·Antigravity·Codex·Copilot 공용).

3. **Claude Code용 링크**: `.claude/skills/wds-*` → `../../.agents/skills/wds-*` 심볼릭 링크(Windows는 복사). Claude Code 플러그인 설치 사용자는 이 단계 생략.

기존 산출물	조치	사유
<code>.cursor/rules/wise-dev-std.mdc</code> (alwaysApply)	삭제 (옵션 <code>--cursor-rule</code> 시에만 glob 규칙으로 생성)	AGENTS.md와 중복
<code>.antigravity/rules.md</code>	삭제 → 필요 시 <code>.agents/rules/wds.md</code> (trigger: <code>always_on</code>)	비공식 경로
<code>.windsurf/rules/...</code>	<code>.devin/rules/...</code> 로 변경, 기본 비활성	AGENTS.md 네이티브 판독
<code>.github/copilot-instructions.md</code>	기본 비활성	Copilot도 AGENTS.md 판독
GEMINI.md	기본 비활성	Antigravity는 AGENTS.md 판독
<code>.clinerules</code> , <code>.roo/rules/</code>	옵션 유지	범위 외 도구 하위호환

표 6-2. standardize 산출물 재정의

6.6 플랫폼별 설치 절차

Claude Code

```
/plugin marketplace add WisemanLim/pub-wise-dev-std
/plugin install wise-dev-std@wise-dev-std
# 위임 대상 공식 Skill (선택)
/plugin marketplace add anthropics/skills
/plugin install document-skills@anthropic-agent-skills
/plugin install example-skills@anthropic-agent-skills
```

Cursor (2.4+)

```
# 방법 1: 프로젝트 설치 (Skill 정보 복사)
bash plugins/wise-dev-std/skills/wds-standardize/scripts/install.sh <project> --target cursor
#   → AGENTS.md + .agents/skills/wds-*/ (Cursor 가 .agents/skills 판독)
# 방법 2: Cursor 플러그인 (.cursor-plugin/plugin.json) 을 Marketplace 또는 /add-plugin 으로 설치
```

Google Antigravity (2.0+)

```
bash plugins/wise-dev-std/skills/wds-standardize/scripts/install.sh <project> --target antigravity
#   → AGENTS.md + .agents/skills/wds-*/ + .agents/agents/stack-architect.md
# 전역 설치: ~/.gemini/config/skills/ | CLI: agy plugin install <path>
```

6.7 Hook 이식

SessionStart 훅(`detect-prd.sh`)은 Claude Code `hooks/hooks.json`과 Cursor `.cursor/hooks.json`(`sessionStart`)에 각각 등록할 수 있다. Antigravity는 SessionStart 이벤트가 없으므로, 동일한 안내("PRD.md가 없으면 wds-prd Skill 사용")를 AGENTS.md의 한 줄 지침으로 대체한다. 훅 스크립트의 `CLAUDE_PROJECT_DIR` 의존은 `${CLAUDE_PROJECT_DIR:-$PWD}`로 이미 풀백되어 있어 재사용 가능하다.

7. 전환 로드맵과 검증 기준

단계	작업	검증 기준 (완료 조건)
Phase 0 정리 (0.5일)	README 버전(0.8.0→1.2.0) · homepage 수정, .antigravity/rules.md 경로 제거 공지	python3 -c "import json..." JSON 검증, README · plugin.json 버전 일치
Phase 1 Skill 통합 (2일)	commands 11개 → skills 통합, wds- 네이밍, 표준 frontmatter + 확장 필드	skills-ref validate skills/* 통과, claude plugin validate 통과, /wise-dev-std:wds-* 11 개 노출
Phase 2 경로 자립화 (2일)	profiles/domains/templates → references · assets 이동, \${CLAUDE_PLUGIN_ROOT} 제거, scripts/ 결정화	grep -r CLAUDE_PLUGIN_ROOT skills/ 0건, scaffold 결과가 전환 전과 diff 동일(13개 프로파일)
Phase 3 공식 위임 (2일)	review · test · implement · ui-design 본문에 공식 Skill 위임 + 풀백	공식 Skill 미설치 환경에서도 동작, 설치 환경에서 위임 호출 로그 확인
Phase 4 멀티 플랫폼 (2일)	wds-standardize 재설계, .cursor-plugin, Antigravity plugin.json, 혹은 이식	Cursor 2.4+ · Antigravity 2.0+에서 /wds-recommend 호출 성공, AGENTS.md 1개만 생성
Phase 5 품질 (1일)	skill-creator eval로 트리거 정확도 측정, /skill-doctor 로 컨텍스트 비용 점검	대표 프롬프트 20개 기준 의도한 Skill 트리거율 ≥ 90%

표 7-1. 단계별 로드맵 (예상 공수는 1인 기준 추정치)

Antigravity Workflow 폐기일(2026-11-01)이 임박해 있으므로, Antigravity 사용자 지원이 필요하다면 Phase 1·4를 우선 진행하는 것이 바람직하다. 플러그인은 현재 Antigravity Workflow를 생성하지 않으므로 폐기 자체의 직접 영향은 없으나, "Antigravity 준용"을 표방하는 이상 Skill 기반 지원이 사실상 유일한 공식 경로가 된다.

8. 위험 요소와 한계

위험	영향	완화 방안
Claude Code의 AGENTS.md 우선순위	CLAUDE.md가 있는 프로젝트에서는 AGENTS.md가 무시됨	standardize가 @AGENTS.md 임포트 줄을 CLAUDE.md에 추가
플랫폼별 Skill 경로 불일치	Claude Code는 .agents/, Antigraivty는 .claude/ 미판독	정본 + 링크/복사 설치 스크립트
확장 필드 비호환	context: fork, disable-model-invocation 등 타 플랫폼 무시	본문만으로 동작하도록 작성, 확장 필드는 최적화 용도
공식 Skill 라이선스	docx/pdf/pptx/xlsx는 source-available	위임만 하고 재배포·번들링 금지
공식 기능 변경 속도	본 문서 기준일 이후 경로·필드 변경 가능	metadata.verified_at 기록, 분기별 재검증 (trends-cache와 동일 주기)
위임에 따른 산출물 형식 차이	.review/ 레이아웃이 공식 리뷰 결과와 상이	고유 산출물(레벨 리뷰·상용화 등급)만 .review/에 유지
Skill 이름 충돌	review, test 등 일반 명칭 충돌	wds- 접두사

표 8-1. 전환 위험 및 완화

8.1 연구의 한계

- 본 분석은 정적 분석과 공식 문서 조사에 기반하며, 각 플랫폼에서 Skill을 실제로 실행한 비교 실험은 포함하지 않는다.
- Cursor의 `.cursorrules` 레거시 상태, Cursor Custom Command의 공식 폐기 여부, Antigraivty Workflow 파일 경로·Knowledge Items 존속 여부, Windsurf→Devin Desktop 리브랜드의 공식 발표는 1차 출처에서 확인되지 않았다.
- Antigraivty 문서의 `/learn` 설명에 `.antigravity/rules.md`가 언급되나 rules 문서와 상충하여, 본 문서는 rules 문서(`.agents/rules/`)를 정본으로 채택했다.

9. 결론

`pub-wise-dev-std`는 Skill 사양(이름 규칙, description 길이, 본문 500줄 미만)을 이미 충족하고 있어 공식 표준 전환 비용이 낮다. 16개 구성요소 중 7개는 공식 대응물이 없는 고유 자산이며, 특히 **PRD 기반 스택 의사결정과 한국 업종 규제 오버레이**는 벤더 공식 Skill로 대체될 수 없는 조직 지식이다. 반면 리뷰·시험·UI·반복 구현·문서 산출 영역의 5개 구성요소는 공식 Skill에 하위 기능을 위임함으로써 유지보수 부담을 줄이고 품질을 플랫폼 개선 속도에 맞출 수 있다.

가장 시급한 변화는 **standardize의 재설계**다. 업계가 AGENTS.md(상시 지침)와 SKILL.md(절차 지식)로 수렴한 현재, IDE별 규칙 파일 복사는 중복과 비공식 경로(`.antigravity/rules.md`)를 낳는다. "AGENTS.md 1개 + Skill 정본 1벌 + 플랫폼별 얇은 패키지"로 전환하면, Cursor·Antigravity 사용자도 규칙 준용을 넘어 `wds-recommend·wds-scaffold·wds-implement`를 Claude Code와 동일하게 실행할 수 있게 된다. 이는 플러그인이 표방한 "Claude CLI 1차 구현, Cursor·Antigravity 준용"을 "세 플랫폼 동등 지원"으로 격상시키는 경로다.

부록 A. Anthropic 공식 Skill 목록 (github.com/anthropics/skills)

Skill	용도	본 플러그인 관련성
docx	Word 문서 생성·편집·분석	리뷰 보고서 산출 위임
pdf	PDF 추출·병합·분할·생성·양식	--pdf 변환 대체
pptx	프레젠테이션 생성·편집	—
xlsx	스프레드시트·수식·서식	시험 결과 집계(선택)
skill-creator	Skill 생성·개선, eval·벤치마크	Phase 5 품질 검증
mcp-builder	MCP 서버 구축 가이드	—
webapp-testing	Playwright 기반 로컬 웹앱 시험	test-runner E2E 위임
frontend-design	의도적 UI 미학, 템플릿 기본값 회피	ui-design 위임
web-artifacts-builder	React/Tailwind/shadcn 다중 컴포넌트 아티팩트	—
doc-coauthoring	문서·제안서·명세 공동작성 워크플로	prd·living-doc 참고 패턴
theme-factory	10종 프리셋 테마 / 신규 테마	ui-design 보조
brand-guidelines	Anthropic 브랜드 색·타이포	—
internal-comms	상태 보고·경영 업데이트	—
canvas-design	포스터 등 정적 시각물	—
algorithmic-art	p5.js 생성 예술	—
slack-gif-creator	Slack용 애니메이션 GIF	—
claude-api	Claude API/SDK 레퍼런스	—
academy-guide	Claude Academy 과정 추천	—
discernment-nudge	답변 후 사실 확인 질문 제시	—

부록 B. 저장소 발견 이슈

#	위치	내용	권고
B-1	README.md:133, README.en.md:132	"현재 0.8.0" 표기 — plugin.json · marketplace.json은 1.2.0	버전 문자열 제거 또는 동기화
B-2	plugin.json homepage, repository	WisemanLim/wise-dev-std(비공개 원본 추정) 지정 — 공개본은 pub-wise-dev-std	공개 저장소 URL로 수정
B-3	standardize, install-portable.sh	.antigravity/rules.md는 Antigravity rules 문서상 공식 경로 아님	.agents/rules/ 또는 AGENTS.md만 사용
B-4	standardize	GEMINI.md의 대상 "Gemini CLI"는 2026-06-18 Antigravity CLI로 대체(무료 티어)	설명 갱신
B-5	standardize	Windsurf → Devin Desktop, .devin/rules/ 우선	경로 갱신
B-6	README § 2 구성 표	living-doc, reverse-prd, ui-design-advisor Skill 누락 (실제 9개, 표 6개)	표 갱신
B-7	README § 4-2 4단계	"다음: standardize"로 안내하나 standardize는 선택 단계	"다음: implement"
B-8	commands + skills	Command-Skill 이중 계층 (Claude Code는 Command를 Skill로 통합)	6장 전환
B-9	skills 4개, commands 4개	\${CLAUDE_PLUGIN_ROOT} 의존 — 타 플랫폼 비호환	상대 경로 + \${CLAUDE_SKILL_DIR}

부록 C. 참고문헌

- [1] Agent Skills Specification. <https://agentskills.io/specification>
- [2] Agent Skills (adopters). <https://agentskills.io> , <https://github.com/agentskills/agentskills>
- [3] Anthropic. Agent Skills overview. <https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview>
- [4] Anthropic. anthropics/skills repository. <https://github.com/anthropics/skills>
- [5] Claude Code Docs — Skills. <https://code.claude.com/docs/en/skills>
- [6] Claude Code Docs — Commands. <https://code.claude.com/docs/en/commands>
- [7] Claude Code Docs — Plugins reference. <https://code.claude.com/docs/en/plugins-reference>
- [8] Claude Code Docs — Plugin marketplaces. <https://code.claude.com/docs/en/plugin-marketplaces>
- [9] Claude Code Docs — Subagents. <https://code.claude.com/docs/en/sub-agents>
- [10] Claude Code Docs — Memory (CLAUDE.md / AGENTS.md). <https://code.claude.com/docs/en/memory>
- [11] Claude Code Docs — Output styles. <https://code.claude.com/docs/en/output-styles>
- [12] Anthropic. claude-code plugins. <https://github.com/anthropics/claude-code/tree/main/plugins>
- [13] Anthropic. claude-plugins-official. <https://github.com/anthropics/claude-plugins-official>
- [14] Cursor Docs — Rules. <https://cursor.com/docs/context/rules>
- [15] Cursor Docs — Skills. <https://cursor.com/docs/context/skills>
- [16] Cursor Changelog 2.4. <https://cursor.com/changelog/2-4>
- [17] Cursor Changelog 1.6. <https://cursor.com/changelog/1-6>
- [18] Cursor Docs — Subagents. <https://cursor.com/docs/context/subagents>
- [19] Cursor Docs — Hooks. <https://cursor.com/docs/agent/hooks>
- [20] Cursor Docs — MCP. <https://cursor.com/docs/context/mcp>
- [21] Cursor Docs — Plugins. <https://cursor.com/docs/plugins>
- [22] Cursor Docs — Bugbot. <https://cursor.com/docs/bugbot>
- [23] Cursor Docs — CLI. <https://cursor.com/docs/cli/using>
- [24] Google. Introducing Antigravity 2.0. <https://antigravity.google/blog/introducing-google-antigravity-2>
- [25] Antigravity Docs — Rules. <https://antigravity.google/docs/rules>
- [26] Antigravity Docs — Workflows. <https://antigravity.google/docs/ide/workflows>
- [27] Antigravity Docs — Skills. <https://antigravity.google/docs/skills>
- [28] Antigravity Docs — Plugins / Hooks / Subagents / MCP. <https://antigravity.google/docs/plugins>
- [29] Antigravity Docs — Slash commands. <https://antigravity.google/docs/slash-commands>
- [30] Antigravity Docs — Gemini CLI migration. <https://antigravity.google/docs/cli/gcli-migration>
- [31] Gemini CLI Docs — Skills. <https://geminicli.com/docs/cli/skills/>
- [32] OpenAI. Build skills (Codex). <https://learn.chatgpt.com/docs/build-skills>
- [33] OpenAI. AGENTS.md (Codex). <https://learn.chatgpt.com/docs/agent-configuration/agents-md>

- [34] GitHub Docs — Repository custom instructions. <https://docs.github.com/en/copilot/how-tos/configure-custom-instructions/add-repository-instructions>
- [35] GitHub Docs — About agent skills. <https://docs.github.com/en/copilot/concepts/agents/about-agent-skills>
- [36] Devin Desktop Docs — Memories/Rules, Skills, Workflows. <https://docs.devin.ai/desktop/cascade/skills>
- [37] Agent Plugins 1.0. <https://agent-plugins.org>, <https://vercel.com/blog/introducing-agent-plugins>
- [38] WisemanLim. pub-wise-dev-std (commit 4be11b9). <https://github.com/WisemanLim/pub-wise-dev-std>